CLOUD-03: AWS EKS Monitoring

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 3 of 8 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

This notebook provides a deep dive into monitoring Amazon Elastic Kubernetes Service (EKS) with Dynatrace. You will learn about EKS-specific monitoring considerations including node groups, Fargate profiles, CloudWatch Container Insights vs Dynatrace, DynaKube operator deployment on EKS, and namespace-level cost allocation patterns.

Table of Contents

1. EKS Monitoring Architecture
2. DynaKube on EKS
3. Node Group Monitoring
4. Fargate Profile Monitoring
5. EKS Metrics with DQL
6. Container Insights vs Dynatrace
7. Namespace Cost Allocation
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail (the DQL cells do not run on Dynatrace Managed)
Permissions	`storage:metrics:read`, `storage:entities:read` + `storage:smartscape:read`, `storage:logs:read`, `storage:buckets:read` (Grail IAM permissions)
AWS EKS Cluster	At least one EKS cluster with Dynatrace Operator installed
Dynatrace Operator	1.8+ (installs the current `v1beta6` DynaKube API; newest release 1.11.1, released 10/22/2026; 1.11.0, released 10/01/2026, removes `v1beta4`) via Helm or kubectl — `v1beta5` with Operator 1.6+ still works and auto-migrates on upgrade
Prior Knowledge	CLOUD-01 fundamentals, basic Kubernetes concepts

1. EKS Monitoring Architecture

Monitoring EKS with Dynatrace involves multiple data paths:

Data Path	Source	What It Monitors
DynaKube Operator	OneAgent on each node	Pods, containers, processes, traces
AWS Cloud Integration	CloudWatch API via the Clouds app connection (or a classic ActiveGate on Managed)	EKS control plane, node group metrics
Kubernetes API	Dynatrace ActiveGate	Cluster events, deployments, workload state
Prometheus Integration	kube-state-metrics, node-exporter	Custom and detailed K8s metrics

EKS-Specific Considerations

- **Control plane is managed** — AWS manages the API server, etcd, and scheduler. You cannot install agents on control plane nodes.
- **Node groups vs Fargate** — Different monitoring approaches for each compute type.
- **VPC CNI** — EKS uses the Amazon VPC CNI plugin, which assigns VPC IP addresses to pods. This affects network monitoring.
- **IRSA (IAM Roles for Service Accounts)** — Recommended for granting DynaKube pods access to AWS resources.

2. DynaKube on EKS

The Dynatrace Operator (DynaKube) is the recommended way to monitor EKS workloads.

Deployment Modes

Mode	Description	Best For
CloudNativeFullStack	Full agent injection + infrastructure monitoring	Production clusters
ApplicationMonitoring	Code-level injection only (no infrastructure)	Shared/multi-tenant clusters
ClassicFullStack	DaemonSet-based (legacy)	Migration from classic deployment

DynaKube Configuration for EKS

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://&lt;environment-id&gt;.live.dynatrace.com/api
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role
          operator: Exists
      nodeSelector:
        kubernetes.io/os: linux
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
```

Key EKS Deployment Notes

- Use **tolerations** to ensure OneAgent runs on all node groups, including tainted nodes
- Enable **kubernetes-monitoring** capability on ActiveGate for cluster-level metrics
- Use **nodeSelector** to exclude Windows nodes if running mixed clusters
- Consider **resource limits** to prevent OneAgent from consuming excessive node resources

> **Sources:** Kubernetes setup (DT docs) — the Operator/DynaKube deployment model, DynaKube parameters (DT docs) — the deployment modes and the API versions (the worked YAML here targets `v1beta6`; K8S-02 § 4 has the served-version table), Operator 1.11.0 release notes (DT docs) — *"The v1beta4 version has been removed from the DynaKube CRD."*, AWS integration (DT docs) — the EKS side of the connection.

Tagging this producer — primary Grail tags on EKS

EKS data comes from two producers, and each needs its own rules:

- **Workloads in the cluster** — Operator and ActiveGate evaluate *Kubernetes* rules at the cluster scope. Enable `metadataEnrichment` in the DynaKube, map namespace labels (for values shared by the whole namespace, such as `app` and `stage`) and workload labels (for values that differ per workload, such as `team`). Workload and pod rule types need Dynatrace Operator 1.11+ and ActiveGate 1.347+ (a staged rollout — check both have reached you). *"Values resolved from Kubernetes metadata are applied when a pod starts, so restart the pods after changing a label."* K8S-10 covers the rule types in detail.
- **The EC2 and EBS resources behind the nodes, and their CloudWatch metrics** — these arrive through the AWS connection, so they take the AWS-account rules in CLOUD-02, not the cluster rules.

Configure each tag at the broadest scope where everything underneath shares one value. FAQ-26 § 5.2 has the Kubernetes options beyond central configuration.

> **Sources:** Apply primary Grail tags across an example AWS estate (DT docs) — *"Values resolved from Kubernetes metadata are applied when a pod starts, so restart the pods after changing a label."* and *"Configure each tag at the broadest level where everything underneath shares one value."*

3. Node Group Monitoring

EKS node groups are collections of EC2 instances that serve as Kubernetes worker nodes.

Node Group Types

Type	Monitoring Approach	Notes
Managed node groups	OneAgent DaemonSet + CloudWatch	Automatic scaling, patching
Self-managed nodes	OneAgent DaemonSet + CloudWatch	Full control, manual management
Karpenter nodes	OneAgent DaemonSet	Dynamic provisioning, varied instance types

Querying EKS Nodes

```
// List Kubernetes cluster entities
fetch dt.entity.kubernetes_cluster, from:-7d
| fields id, entity.name, tags
| sort entity.name asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fieldsKeep id, name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via
getNodeField).
```

Node CPU and Memory Usage

```
// Node-level CPU usage over the last hour, derived at container grain.
//
// `dt.kubernetes.node.cpu_usage` and `dt.kubernetes.node.memory_working_set`
DO NOT EXIST
// (verified 08/12/2026 against a full 836-key catalog enumeration). The
`dt.kubernetes.node.*`
// namespace is real, but it publishes CAPACITY and CONDITION metrics —
`cpu_allocatable`,
// `memory_allocatable`, `pods_allocatable`, `conditions` — not utilization.
Node-level
// utilization is derived by summing the container metrics per node, which is
what this cell does.
// A timeseries against a missing key returns an EMPTY result rather than an
error, so the old
// cell simply drew nothing. Enumerate the real catalog with:
//   metrics | filter startsWith(metric.key, "dt.kubernetes") | summarize n =
count(), by:{metric.key} | sort metric.key asc
// (`metrics` takes `from:` with NO leading comma; `summarize` requires an
aggregation, and a
//  bare `metrics | fields metric.key` is capped and will silently under-
report the catalog.)
timeseries used = sum(dt.kubernetes.container.cpu_usage, rollup: avg),
from:-1h, by:{dt.entity.kubernetes_node}
| fieldsAdd avgCpu = arrayAvg(used)
| sort avgCpu desc
| limit 15
```

```
// Node memory working set over the last hour, derived at container grain.
//
// `dt.kubernetes.node.cpu_usage` and `dt.kubernetes.node.memory_working_set`
DO NOT EXIST
// (verified 08/12/2026 against a full 836-key catalog enumeration). The
`dt.kubernetes.node.*`
// namespace is real, but it publishes CAPACITY and CONDITION metrics —
`cpu_allocatable`,
// `memory_allocatable`, `pods_allocatable`, `conditions` — not utilization.
Node-level
// utilization is derived by summing the container metrics per node, which is
what this cell does.
// A timeseries against a missing key returns an EMPTY result rather than an
error, so the old
// cell simply drew nothing. Enumerate the real catalog with:
//   metrics | filter startsWith(metric.key, "dt.kubernetes") | summarize n =
count(), by:{metric.key} | sort metric.key asc
// (`metrics` takes `from:` with NO leading comma; `summarize` requires an
aggregation, and a
//  bare `metrics | fields metric.key` is capped and will silently under-
report the catalog.)
//
// The node-CPU sibling was corrected on 08/11/2026 and this cell was missed,
so it kept drawing
// an empty tile for a further day — when a key turns out not to exist, sweep
the whole namespace.
timeseries nodeMemory = sum(dt.kubernetes.container.memory_working_set,
rollup: avg), from:-1h, by:{dt.entity.kubernetes_node}
| fieldsAdd avgMemory = arrayAvg(nodeMemory)
| sort avgMemory desc
| limit 15
```

4. Fargate Profile Monitoring

AWS Fargate runs pods without managing nodes. This changes the monitoring approach significantly.

Fargate Monitoring Limitations

Capability	EC2 Nodes	Fargate
OneAgent DaemonSet	Supported	Not supported (no node access)

Capability	EC2 Nodes	Fargate
Code injection	Via DaemonSet	Via init container (ApplicationMonitoring mode)
Host metrics	Full (CPU, memory, disk, network)	Container metrics only
Process monitoring	Full	Limited
Network monitoring	Full	Limited

Monitoring Fargate Workloads

For Fargate pods, use **ApplicationMonitoring** mode in DynaKube:

```
spec:
  oneAgent:
    applicationMonitoring: {}
```

> **Note:** `useCSIDriver` is **not** a DynaKube `v1beta5` / `v1beta6` field — for those versions the `.spec.oneAgent.applicationMonitoring` parameters are `additionalResourceAttributes`, `codeModulesImage`, `codeModulesImagePullPolicy`, `imagePullPolicy`, `initResources`, `namespaceSelector` and `version` (`useCSIDriver` appears only under the retired `v1beta1` / `v1beta2` APIs). Whether the CSI driver runs is decided when you **install the Operator**, not in the DynaKube. The driver is optional and runs as a node DaemonSet, which Fargate cannot host — so install the Operator with the **Without CSI driver** variant of the application-observability setup guide. For the manifest install that is `kubernetes.yaml` rather than `kubernetes-csi.yaml`:
>
> `bash > kubectl apply -f https://github.com/Dynatrace/dynatrace-operator/releases/download/v1.10.2/kubernetes.yaml >`
>
> For a Helm install, the same guide sets `--set "csidriver.enabled=false"` on `helm install dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-operator`.
>
> The guide pinned `v1.10.2`, the current release on 09/24/2026; 1.11.0 followed on 10/01/2026 — substitute the Operator version you run.

> **Sources:** DynaKube parameters (DT docs) — the per-version `applicationMonitoring` parameter tables, Application observability setup (DT docs) — *"CSI driver is optional (see Step 2). If enabled, it gets deployed as DaemonSet and results in a CSI driver Pod on each node."*; Helm tab: `--set "csidriver.enabled=false"`, Fargate considerations (AWS docs) — *"Daemonsets aren't supported on Fargate."*

5. EKS Metrics with DQL

Pod CPU and Memory by Namespace

```
// Namespace CPU: SUM of container CPU per namespace (avg would report the
per-container mean)
timeseries nsCpu = sum(dt.kubernetes.container.cpu_usage, rollup: avg),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(nsCpu)
| sort avgCpu desc
| limit 10
```

```
// Namespace memory: SUM of container working set per namespace (avg would
report the per-container mean)
timeseries containerMem = sum(dt.kubernetes.container.memory_working_set,
rollup: avg), from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgMem = arrayAvg(containerMem)
| sort avgMem desc
| limit 10
```

Pod Status Overview

```
// Kubernetes events by reason over the last 6 hours.
//
// Corrected 08/11/2026 — this cell previously carried three separate filters
that each
// matched nothing, so it returned zero rows against an estate emitting
thousands:
//   1. `event.kind == "K8S_EVENT"` — event.kind never takes that value.
Verified over 24h
//      the only values are DAVIS_EVENT, SYNTHETIC_EVENT, FLEET_EVENT and
DAVIS_PROBLEM.
//      Kubernetes events arrive as DAVIS_EVENT; the discriminator is
event.provider.
//   2. `event.type == "Warning"` — every KUBERNETES_EVENT record carries
CUSTOM_INFO
//      (3,571 of 3,571 checked). The Warning/Normal split is in status
("WARN" / "INFO");
//      dt.kubernetes.event.important was "true" on every event over 30 days
(10/02/2026).
//   3. `event.reason` — null on every record. The real field is
dt.kubernetes.event.reason.
// Each filter was valid syntax that executed cleanly, which is why this
survived review.
// Counted with countDistinctExact(event.id): an open Kubernetes event is re-
written about every
// 5 minutes under the same event.id, so count() counts refresh records
(10/15/2026: 1,203
// distinct events vs 8,259 records in 24 h).
fetch events, from:-6h
| filter event.provider == "KUBERNETES_EVENT"
| filter status == "WARN"
| summarize event_count = countDistinctExact(event.id), by:
{dt.kubernetes.event.reason, k8s.namespace.name}
| sort event_count desc
| limit 20
```

6. Container Insights vs Dynatrace

Many EKS users start with CloudWatch Container Insights. Here is how it compares to Dynatrace:

Capability	CloudWatch Container Insights	Dynatrace
Setup complexity	AWS-native, minimal	DynaKube operator deployment
Metric granularity	1-minute	10-60 seconds
Distributed tracing	X-Ray integration	Built-in PurePath
AI-powered root cause	Basic alarms	Dynatrace Intelligence automatic root cause
Code-level visibility	None	Full (method-level)
Log analysis	CloudWatch Logs Insights	Grail-powered DQL
Cost model	Per metric, per log GB	DPS/DDU-based
Cross-stack correlation	Limited	Automatic (host → process → service → trace)

When to Use Both

- Keep Container Insights for **EKS control plane logging** (API server, authenticator, scheduler)
- Use Dynatrace for **workload monitoring**, **distributed tracing**, and **AI-powered alerting**
- Forward Container Insights logs to Dynatrace via **CloudWatch log forwarding** for unified analysis

7. Namespace Cost Allocation

One of the key benefits of Kubernetes monitoring is the ability to allocate costs by namespace, team, or application.

CPU-Based Cost Allocation by Namespace

```
// Namespace CPU usage (sum over the namespace's containers) over the last 24 hours
// rollup: avg averages each container WITHIN a time bucket before summing across containers.
// Without it, sum() also adds up the 1-minute points in each bucket: at 24h (10-min buckets)
// every value reads ~10x high (verified 09/25/2026: 19,512 vs 1,955 millicores for one namespace).
timeseries nsCpu = sum(dt.kubernetes.container.cpu_usage, rollup: avg), from:-24h, by:{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(nsCpu)
| sort avgCpu desc
| limit 15
```

Memory-Based Cost Allocation by Namespace

```
// Namespace memory usage over the last 24 hours
// rollup: avg averages each container WITHIN a time bucket before summing across containers.
// Without it, sum() also adds up the 1-minute points in each bucket: at 24h (10-min buckets)
// every value reads ~10x high (verified 09/25/2026: 19,512 vs 1,955 millicores for one namespace).
timeseries nsMem = sum(dt.kubernetes.container.memory_working_set, rollup: avg), from:-24h, by:{k8s.namespace.name}
| fieldsAdd avgMemBytes = arrayAvg(nsMem)
| fieldsAdd avgMemGB = avgMemBytes / 1073741824.0
| sort avgMemGB desc
| limit 15
```

Cost Allocation Best Practices

Practice	Description
Enforce namespace labels	Require `team`, `cost-center`, `environment` labels on all namespaces
Set resource requests/limits	Accurate requests enable fair cost attribution
Use Dynatrace segments	Create segments per team/namespace for filtered dashboards

Practice	Description
Track over time	Use `makeTimeseries` to trend namespace costs weekly

8. Summary and Next Steps

Key Takeaways

- EKS monitoring requires **DynaKube Operator** for workload visibility and **cloud integration** for control plane metrics
- **Fargate** pods need ApplicationMonitoring mode with the Operator installed **without the CSI driver** (`useCSIDriver` is not a DynaKube `v1beta5` / `v1beta6` field)
- Dynatrace provides deeper visibility than CloudWatch Container Insights, especially for **distributed tracing** and **AI root cause analysis**
- **Namespace-level metrics** enable team-based cost allocation

Next Steps

- **CLOUD-04: AWS Lambda & Serverless** — Monitoring serverless workloads
- **CLOUD-07: CloudWatch Log Ingestion** — Forwarding EKS logs to Dynatrace
- See the **K8S notebook series** for general Kubernetes monitoring patterns

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.